

Executive Summary

This project aims to compare **GARCH-EVT**, **Realized GARCH**, and **Quantile Regression** approaches for forecasting volatility and tail risk during extreme market events. We divide responsibilities between two research programmers (A and B), who will jointly acquire and preprocess data, implement models, run rolling forecasts, and evaluate performance. Key steps include data acquisition (daily & intraday), realized-volatility construction, baseline GARCH modeling, two-stage GARCH-EVT, Realized GARCH, and quantile regression implementations, rolling-window forecasting, and comprehensive evaluation (volatility errors, VaR/ES backtesting, regime analysis, robustness checks, statistical tests). The plan includes clear milestones, deliverables, and a Gantt timeline (below) spanning approximately 4–5 months. All code will be version-controlled, reproducible, and handed off to the writing team with documentation.

Key components: Data sourcing (e.g. NYSE TAQ for intraday [14]), model coding (Python/R with `arch`, `statsmodels`, etc.), rolling OOS engine, metrics (RMSE, QLIKE, Kupiec/Christoffersen tests), robustness (thresholds, windows), crisis/subperiod analysis, and team responsibilities.

Team & Responsibilities

- **Researcher A:** Leads **data processing**, baseline GARCH coding (GARCH(1,1), GJR/EGARCH), Realized GARCH implementation, rolling-forecast engine; supports EVT and QR coding; leads environment setup and reproducibility.
- **Researcher B:** Leads **GARCH-EVT** and **quantile regression** model implementation, evaluation metrics (VaR/ES tests), crisis/regime analysis; supports baseline GARCH and Realized GARCH coding; leads statistical comparison (Diebold–Mariano, etc.) and result visualization.
- **Both** collaborate on testing, documentation, figure generation, and code reviews. (Neither is responsible for final writing – deliverables handed to writing team.)

Each task will have a primary lead and a secondary support. Below is a summary of roles:

Task Area	Lead (A/ B)	Support	Estimated Hours (Team)
Data acquisition (daily & intraday) & cleaning	A	B	40 h
Realized volatility construction	A	B	16 h
Baseline GARCH & GJR/EGARCH coding	A	B	24 h
GARCH-EVT (filtering + EVT tail fitting)	B	A	24 h
Realized GARCH model coding	A	B	24 h
Quantile regression (with predictors)	B	A	16 h
Rolling out-of-sample forecast engine	A	B	24 h

Task Area	Lead (A/ B)	Support	Estimated Hours (Team)
Evaluation metrics (RMSE, QLIKE, VaR/ES backtest)	B	A	24 h
Crisis/regime analysis	B	A	16 h
Robustness checks (thresholds, windows, horizons)	A	B	16 h
Statistical tests (Diebold–Mariano, etc.)	B	A	12 h
Figures & tables preparation	A/B	each other	16 h
Code review & reproducibility (Docker/env)	A/B	each other	16 h
Total (approx)			272 h

All work will be tracked in the project repo. Code review checkpoints will occur after each major module (e.g. after data preprocessing, after model implementations, before final evaluation). A final reproducibility audit (using fixed random seeds and an environment file) will precede handoff to the writing team.

Data Sources & Preprocessing

1. **Assets/Indices:** Select 1–3 major indices (e.g. S&P 500, NASDAQ-100, maybe a commodity or FX pair for robustness). Use *official* or high-quality sources:
2. **Daily prices:** (Open/High/Low/Close) from data vendors (e.g. Bloomberg, Refinitiv, or free sources like Yahoo Finance or Quandl). Cite data vendor documentation if possible.
3. **Intraday data:** Use high-frequency trade/quote data (e.g. **NYSE TAQ** via WRDS [14], which covers 10,000+ stocks on 16 US exchanges). Alternatively consider commercial feeds (e.g. Bloomberg tick, Thomson Reuters). If needed for a single index, we can simulate intraday by using SPY ETF tick data (e.g. IEX or Plexus API).
4. **Risk indicators:** VIX (CBOE) or similar can be added as exogenous variable for quantile models.
5. **Coverage:** At least 10+ years to include multiple crises (e.g. 2008 GFC, 2011 Euro debt, 2020 Covid). For intraday, a subset (e.g. 5-min or 15-min data) may suffice to compute realized volatility.
6. **Licensing:** Note that official tick data often require subscriptions (TAQ on WRDS [14] is common for academics). Quandl or Yahoo are public but limited. Document any usage restrictions.
7. **Preprocessing tasks (A leads, B supports; ~40h):**
8. **Cleaning:** Check for missing days, corporate actions, holidays. Align daily and intraday timestamps.

9. **Outliers:** Winsorize or filter erroneous ticks.
10. **Returns:** Compute log returns from prices (daily and intraday).
11. **Realized Volatility:** From intraday returns, compute daily realized variance: e.g. $RV_t = \sum_{i=1}^M r_{t,i}^2$ for the day (M intra intervals). This will feed Realized GARCH and possibly quantile regressors.
12. **Crisis labels:** Define “event” periods by dates (e.g. 2008-2009, 2010-11, 2020, etc.) or volatility thresholds.

Deliverable: Cleaned datasets (daily returns series, realized volatility series, VIX, etc.) stored as CSV/Parquet files in the repo. Ensure reproducible scripts for ingestion (Jupyter or .py).

Table: Data Assets and Sources

Data	Source / Provider	Frequency	Date Range	Notes
S&P 500 Index (or ETF) prices	Yahoo Finance / Quandl / Bloomberg	Daily	2010–2025 (e.g.)	Free/official API
S&P 500 Intraday (TAQ)	NYSE TAQ via WRDS [14]	Tick/1-min	2010–2020	Academic licensed
VIX Index	CBOE (via CBOE or YF)	Daily	2010–2025	Market fear gauge
Other indices/commodities	Bloomberg / Quandl	Daily	2010–2025	e.g. NASDAQ, Gold

Model Implementation

Researchers will code all models in a common language (Python recommended, but R is acceptable). We suggest **Python** with these libraries: `pandas`, `numpy` (data); `arch` (GARCH models, VaR tests) [11†L41-L49] ; `statsmodels` (quantile regression); `scipy` (EVT); `matplotlib/seaborn` (plots); `sklearn` (possibly for DM test); and `pip` / `conda` for package management. Example setup:

```
conda create -n volproj python=3.10
pip install pandas numpy scipy arch statsmodels matplotlib seaborn plotly
```

Alternatively, if using R: `rugarch`, `evd/POT`, `quantreg`, `PerformanceAnalytics`. No constraint given, so either is fine.

3.1 Baseline GARCH Models (A leads, 24h):

- Fit GARCH(1,1) on daily returns. Also fit an asymmetric variant (GJR-GARCH or EGARCH) to capture leverage effects (as per Engle & Ng, 1993). Use maximum likelihood (Arch library or rugarch). Validate in-sample fit.
- **Deliverable:** Code that outputs fitted GARCH residuals and forecasts (volatility) for each day. Document parameter estimates.

3.2 GARCH-EVT Two-Stage (B leads, 24h):

- **Stage 1:** Use the baseline GARCH filter to obtain standardized residuals $z_t = \epsilon_t / \sigma_t$.
- **Stage 2:** Fit a **Generalized Pareto Distribution (GPD)** to the tails of $\{z_t\}$ above a high threshold u . This follows McNeil & Frey (2000) style: conditional EVT for exceedances **【5†L1-L4】** **【4†L18-L25】**. Use `scipy.stats.genpareto` or POT package.
- Calibrate threshold u (e.g. 95th percentile of $|z|$) via QQ-plot.
- Combine GARCH and EVT to compute dynamic VaR and ES (e.g. 1% VaR = $\sigma_t \times u$ + **GPD tail**).
- **Deliverable:** Functions to fit GPD, compute VaR/ES forecasts. Document threshold choice and tail index.

3.3 Realized GARCH (A leads, 24h):

- As in Hansen & Huang (2010) **【11†L31-L39】** **【11†L41-L49】**, jointly model returns and realized variance. E.g. specify a Realized GARCH(1,1) with log-link:

$$r_t = \mu + \epsilon_t, \quad \epsilon_t = \sigma_t z_t; \quad \log(\sigma_t^2) = \omega + \beta \log(\sigma_{t-1}^2) + \alpha \log(RV_{t-1})$$

plus a measurement equation linking observed RV_t to latent σ_t^2 .

- Use existing implementations if available (e.g. the “arch” package has a Realized GARCH class, or custom code). Incorporate high-frequency RV_t from preprocessing.
- **Deliverable:** Estimation script for Realized GARCH, with outputs of volatility forecasts.

3.4 Quantile Regression (B leads, 16h):

- Specify quantile regression models for conditional returns (or volatility) at tail levels (e.g. 1%, 5%). Possible predictors: lagged returns, lagged volatility (from GARCH), realized volatility, VIX, market factor returns, etc.
- Use `statsmodels.QuantReg` (Python) or `quantreg` (R). Estimate separate regressions for each quantile (0.01, 0.025, 0.05) **【16†L51-L59】**.
- Optionally include lags of predictors and experiment with different combinations.
- **Deliverable:** Scripts to train quantile models and output 1-step-ahead VaR forecasts. Ensure regression outputs (coeffs, in-sample fit) are logged.

Table: Model Specifications

Model	Inputs	Main Output	Notes
GARCH(1,1)	Daily returns	Conditional volatility forecast	Baseline (Bollerslev, 1986)
GJR-GARCH	Daily returns (with leverage term)	Volatility forecast (asymmetric)	Captures volatility spikes
GARCH-EVT	GARCH residuals + EVT tail fit	VaR / ES forecasts	Two-stage (McNeil & Frey, 2000)

Model	Inputs	Main Output	Notes
Realized GARCH	Returns + realized variance (RV)	Volatility forecast + VaR	Hansen & Huang (2010) framework 【11†L31-L39】 【11†L41-L49】
Quantile Reg.	Returns, RV, VIX, lags, etc.	Conditional quantile forecasts (VaR)	No distributional assumption 【16†L51-L59】

Forecasting & Evaluation

4.1 Rolling Out-of-Sample Engine (A leads, 24h): Implement a sequential forecasting loop for fair comparison:

- **Setup:** Define an initial estimation window (e.g. first 2000 days) and holdout period. For each step t from the start of holdout to end of sample:
- Re-fit each model on data up to $t - 1$ (with fixed window or expanding window).
- Produce 1-day-ahead forecasts (volatility, VaR, ES as appropriate).
- Record forecasts for later evaluation.

This can be parallelized (e.g. joblib) but ensure reproducibility (fixed random state). Store outputs (time series of forecasts and actual returns).

- **Deliverable:** Forecast pipeline code and a consolidated forecast dataset (predicted vs actual).

4.2 Evaluation Metrics (B leads, 24h):

- **Volatility forecasts:** Compute error metrics against “realized volatility” proxy (e.g. future squared return or RV). Use **RMSE**, **MAE**, and **QLIKE** (the quasi-likelihood loss 【11†L41-L49】). Plot predicted vs realized series.
- **VaR/ES performance:** For each model’s VaR level, conduct backtests: Unconditional coverage (Kupiec 1995), Conditional coverage (Christoffersen 1998), and the Dynamic Quantile (DQ) test. Evaluate Expected Shortfall (use e.g. Acerbi-Székely or backtest scoring).
- Document the proportion of violations and test p-values. Use `arch` library’s `losses` or custom code for these tests.

Include these metrics **overall** and by **regime** (see next).

4.3 Crisis/Regime Analysis (B leads, 16h):

- Define market regimes by date (normal vs crisis) or by volatility thresholds.
- Compute each metric **subsample** (e.g. during 2008-09, 2020).
- Create **regime-conditional** plots: e.g. bar charts of RMSE or VaR violation rates in crisis vs normal. This highlights model performance shifts.

Figures & Tables: Prepare essential visualizations:

- **Volatility Forecast Plots:** Predicted vs realized vol (whole sample and zoom-ins on crises).
- **VaR Exceedance Plots:** Time series showing actual breaches vs VaR lines; Basel traffic-light table.

- **Performance Heatmaps:** RMSE/QLIKE/VaR p-values for each model across regimes.
- **Regime Comparison Charts:** Bar/line charts of metrics during crisis vs calm.
- **Table of Model Comparison:** Summarize key stats (avg. RMSE, %VaR breaches, DM test results).

These figures (using matplotlib/seaborn) will be handed off to the writing team.

Table: Deliverables & Milestones

Deliverable	Description	Due (Week)	Lead
Clean data files & code	Daily/intraday data prepared; ingestion scripts	2	A
Realized volatility series	Computed from intraday data	3	A
Baseline GARCH code	GARCH(1,1) and GJR/EGARCH implementations	4	A
GARCH-EVT code	Two-stage EVT VaR/ES estimation scripts	5	B
Realized GARCH code	Joint returns-RV model implementation	6	A
Quantile regression code	Scripts estimating VaR quantiles	6	B
Rolling forecast engine	Combined forecasting loop (all models)	8	A
Evaluation scripts	Metrics (RMSE, QLIKE, VaR tests)	9	B
Initial results & figures	Draft plots/tables of early findings	10	A/B
Regime-specific analysis	Crisis vs normal performance tables/plots	11	B
Robustness analysis	Vary thresholds, windows, produce summary results	12	A
Statistical comparison tests	Diebold–Mariano and others to compare models	12	B
Code repository & environment	Finalized codebase, README, requirements.txt	13	A/B
Reproducibility audit report	Confirm fixed seeds, environment file (Docker)	14	A/B
Handoff package for writers	Figures, tables, and documentation summary	15	A/B

(Weeks are approximate; see Gantt chart below for timeline.)

Evaluation & Robustness

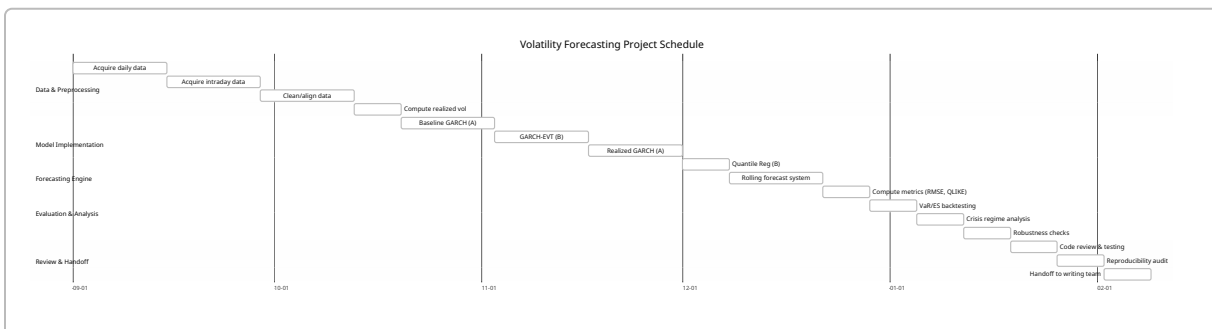
- **Forecast comparison:** Use statistical tests (Diebold–Mariano) to test if differences in forecast accuracy are significant [18†L0-L2] .
- **Threshold sensitivity:** In GARCH-EVT, vary EVT threshold (e.g. 90%, 95%, 99%) and check how VaR/ES forecasts change.

- **Window length:** Try different rolling window sizes (e.g. 2 vs 5 years) and expanding windows as robustness.
- **Horizon:** If applicable, extend forecasts to 5-day ahead and compare results.
- **Cross-asset checks:** If time permits, repeat the pipeline on a second index/commodity to ensure results are not asset-specific.

Reproducibility & Environment

- **Version Control:** All code in a Git repo (GitHub/Bitbucket). Use branches for major features.
- **Reproducibility:** Use a fixed random seed for simulations/optimizations. Include an environment spec (`requirements.txt` or `environment.yml`). Consider a Dockerfile or container for the final environment.
- **Testing:** Write unit tests for core functions (e.g. GPD fit, quantile regression prediction). Document how to run tests.
- **Documentation:** Inline comments and a README outlining how to run each step. Ensure the timeline/milestones are documented in the repo's wiki or README.

Timeline (Gantt Chart)



Visualizations

Planned charts include: **Forecast vs Realized Volatility** (lines), **VaR Exceedance** (step or bar charts of violations), **Heatmaps of model errors/RMSE** across regimes, and **Bar charts** showing tail-risk metrics by model/regime. We will also create tables summarizing model parameters, performance metrics, and statistical test results. Key plots might resemble volatility forecasts over time with crisis highlights or “traffic-light” graphs of VaR breaches.

Sources: Seminal methods are from Bollerslev (1986) for GARCH, McNeil & Frey (2000) for GARCH-EVT, Hansen & Huang (2010) for Realized GARCH [11†L31-L39] [11†L41-L49] , and Koenker (1978) for quantile regression [16†L51-L59] . Intraday data (TAQ) is described by NYSE [14]. Each model's implementation will follow these established frameworks.